

Affine-inspired encoder-decoder models for risk-neutral term structure modeling*

Rastin Matin[†] Wenjie Wan[‡] Rolf Poulsen[§]

September 18, 2025

1 Introduction

An encoder-decoder model is a neural network architecture designed to learn a compressed representation of input data. It consists of two main components: An encoder which maps the input to a lower-dimensional latent space, and a decoder which reconstructs the original input – and possibly more – from this compressed representation. Such non-linear or non-parametric dimensionality reduction fits hand-in-glove for modeling curve dynamics in finance as pointed out by [Kondratyev \(2018\)](#) and [Sokol \(2022\)](#) among others. These models are not arbitrage-free by default though, so additional constraints must be imposed if they are to be used for risk-neutral modeling.

The mother of all financial curves is the discount function, which tells us the value today (t) of receiving 1 unit at the future date $T = t + \tau$, where we call τ (time to) maturity. This may sound trivial, but things move stochastically with time t , and there are many τ s, so it very much isn't. Term structure theory and models tell us how to handle this. It gives us the necessary no-arbitrage relations between the current yield curve, its dynamics and possible future yield curves.

Encoder-decoder models for yield curve modeling often take as input a set of swap rates observed at discrete maturities and produce a low-dimensional latent representation. From this a continuous curve for the instantaneous forward rate is extracted using a curve

*The authors thank Morten Barslund, Mads Gydesen and David Skovmand for helpful comments. The views expressed in this paper are those of the authors and do not necessarily reflect the positions of their employers or affiliates.

[†]PFA Asset Management. rtm@pfa.dk

[‡]wnw19940819@gmail.com

[§]Department of Mathematical Sciences, University of Copenhagen. rolf@math.ku.dk

construction method. The works of Sokol (2022) and Andreassen (2023) are examples of this. Andreassen (2023) for instance imposes the no-arbitrage condition at the decoding stage as a soft constraint via a regularization term, and subsequently employs a spline interpolator as curve constructor. The generated yield curves are only approximately arbitrage-free on this finite set of points due to the rudimentary nature of the spline interpolator.

Building on these insights, we impose the no-arbitrage condition at the decoding stage as a hard constraint. This enables us to specify and train a model in which the encoder-decoder *itself* produces a continuous representation that is *exactly* arbitrage-free by construction.

The novelty of an encoder-decoder that directly produces a continuous representation is also explored in the recent work of Lyashenko et al. (2025). There, the Musiela-parameterized instantaneous forward rate curves are written as $f(t, \tau) = \hat{f}_Z(Z(t), \tau)$ where Z is a latent low-dimensional factor process and $\hat{f}_Z$ is what the authors call the autoencoder.¹ They look for decoders with functional forms that satisfy the no-arbitrage condition, initially focusing on those with the so-called time-shift invariance property, and propose parametric forms that facilitate efficient model training. However, as the authors acknowledge, real-world yield curves may not be well captured by such autoencoders. To address this, they introduce a convexity adjustment on top of a base arbitrage-free autoencoder that can be calculated explicitly given the base model. Our approach departs from this framework: We decouple the encoding and decoding stages and, by solving a system of ordinary differential equations (ODEs) for every encoded latent factor, obtain a continuous zero-coupon bond price curve that satisfies the no-arbitrage condition. Our decoder is therefore guaranteed to produce arbitrage-free yield curves without relying on approximations or post hoc adjustments, but comes with the additional computational cost of solving as many ODEs as the number of input par swap rates. So while the objectives are the same, the two approaches are fundamentally different.

2 Encoder-Decoder Models for Swap Curves

As noted in the introduction, we decouple the encoding and decoding stages and enforce the no-arbitrage condition at the latter as a hard constraint. In the following sections we describe both stages and explain in detail the mathematical formulation used to enforce the no-arbitrage condition.

¹An autoencoder is a special case of an encoder-decoder, where the input and output domains are the same (Minaee et al., 2022).

2.1 Linear Encoding

The overall architecture of our model is illustrated in Figure 1. Denoting the input and output layer by n_0 and n_3 , respectively, the model contains two hidden layers n_1 and n_2 . The input $x_0(t)$ at n_0 is the set of observed par swap rates S with discrete maturities

$$\{1Y, 2Y, 3Y, 5Y, 10Y, 15Y, 20Y, 30Y\} \quad (1)$$

Yield curves such as these are known for having low dimensionality, meaning they can be represented with few latent variables as noted by Sokol (2022). Our model subsequently employs only two latent factors $z_t \equiv (z_1(t), z_2(t))$ for the compressed representation of the input, which our encoder generates using a linear transformation

$$z_t = A_1 x_0(t) \quad (2)$$

Here A_1 is a matrix that maps $x_0(t)$ from $\mathbb{R}^8$ to z_t in $\mathbb{R}^2$. The latent factors z_t can be decoded back to par swap rates S by using zero-coupon bond prices. Ignoring the basis between forward rates and discounting, we compute the estimated swap rates $\tilde{S}$ as (see, e.g., Henry-Labordere (2008), Björk (2009) or Andreasen (2023))

$$\tilde{S}(z_t, \tau) = \frac{1 - P(z_t, \tau)}{\sum_{\tau' \leq \tau} P(z_t, \tau')} \quad (3)$$

where τ is a maturity on the annually-spaced grid

$$\tau \in \{1Y, 2Y, 3Y, \dots, 29Y, 30Y\} \quad (4)$$

and $P(z_t, \tau)$ denotes the time- t price of zero-coupon bonds on this grid. For clarity, we note that the sum in (3) runs over the maturities in (4) smaller than or equal to τ .

2.2 Arbitrage-Free Decoding

Now that the encoder has been established, we turn our attention to the decoder. Guided by performance metrics, we determine empirically that two hidden layers with dimensions $(n_1, n_2) = (2, 10)$ are optimal. Empirical optimization is a common approach when designing neural networks since training is influenced by various factors such as input data, architecture, loss function, and optimization algorithm (Jiang et al., 2024). This is also why our architecture dimensions differ from those used in related work such as

Andreasen (2023). We furthermore use a centered soft step function defined by

$$\psi(x) = \frac{1}{1 + e^{-x}} - \frac{1}{2} \quad (5)$$

as the activation function.

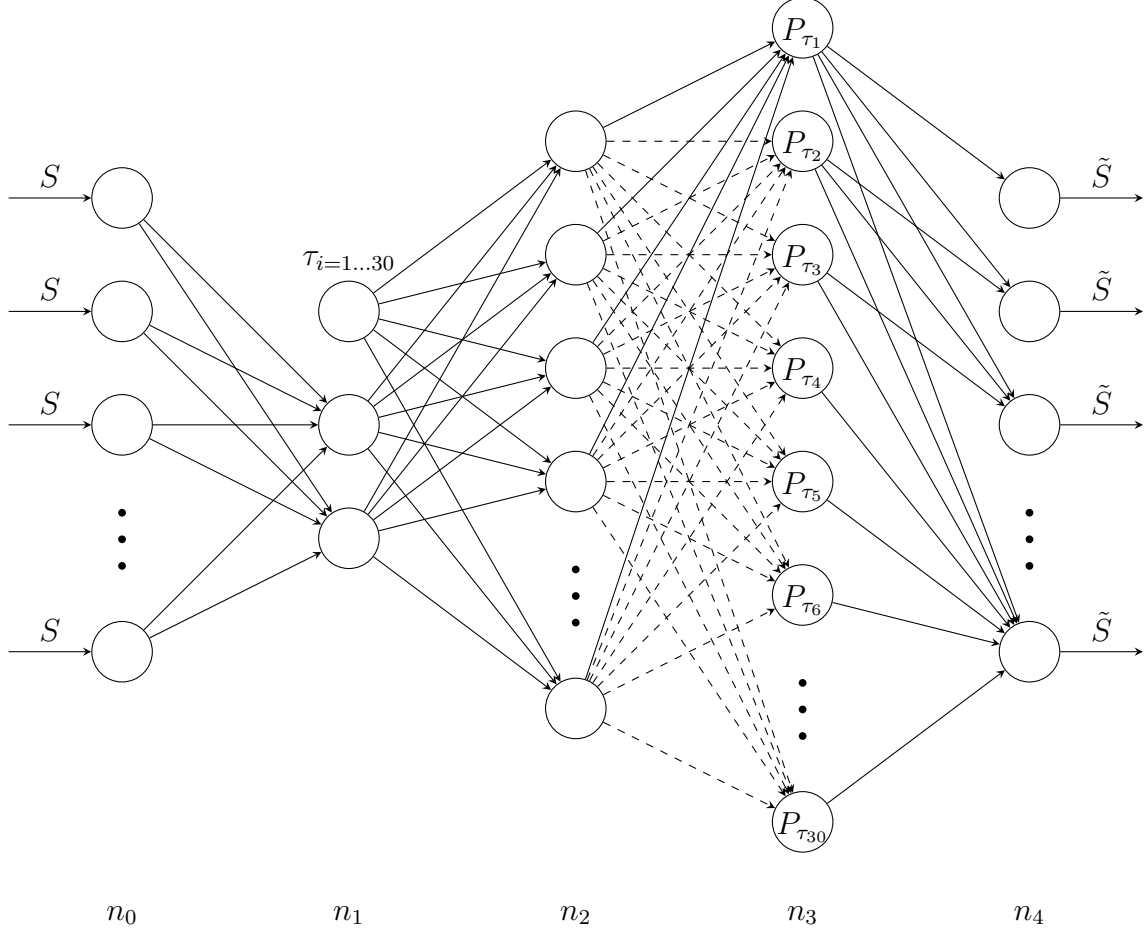


Figure 1: Architecture of the affine-inspired encoder-decoder model for par swap rates. The input S is the set of observed par swap rates and the output $\tilde{S}$ is the set of estimated par swap rates; they share the same discrete maturities $\{1Y, 2Y, 3Y, 5Y, 10Y, 15Y, 20Y, 30Y\}$. The encoder has dimensions $(n_0, n_1) = (8, 2)$ while the decoder has dimensions $(n_1, n_2, n_3) = (3, 10, 1)$ from which a zero-coupon bond price P_{τ_i} is generated for a given maturity τ_i on the annually-spaced grid $\{1Y, 2Y, 3Y, \dots, 29Y, 30Y\}$. The generated zero-coupon bond prices are then converted to par swap rates at n_4 using (3). The combination of dashed and solid lines from n_2 to n_3 illustrates that the decoding from latent factors to zero-coupon bond prices is a sequential process in which only one price is generated at each step. The transformation to par swap rates $\tilde{S}$ begins once all zero-coupon bond prices have been generated on the grid.

A few remarks are in order. As illustrated in Figure 1, we include the maturity τ in n_1 such that there are 3 neurons in total. This τ -dependency effectively turns the decoder into a continuous curve constructor so we – in contrast to Andreassen (2023) for instance – avoid the need for an explicit curve constructor after the decoder. Embedding the curve constructor in the decoder like this is sensible as neural networks are good at generating smooth functions – a common requirement for a yield curve constructor – and allows us to design a more concise architecture with fewer parameters. The output at n_3 is therefore a continuous zero-coupon bond curve and is a scalar for a given value of τ . When decoding the latent factors z_t to par swap rates, we follow the recipe outlined by (3), and the estimated par swap rates $\tilde{S}$ therefore have the same dimensions as the input par swap rates S as illustrated in the figure.

We now describe the model governing zero-coupon bond prices at n_3 . We draw inspiration from the classical affine model of Duffie and Kan (1996), where a zero-coupon bond with maturity τ is expressed in terms of a vector of latent variables x

$$P(x, \tau) = \exp\left(A(\tau) + \sum_i B_i(\tau)x_i\right) \quad (6)$$

The scalar functions $A(\tau)$ and $B_i(\tau)$ can be obtained by solving a system of coupled ODEs. We take a neural leap and replace $\sum_i B_i(\tau)x_i$ with a scalar function $B_{z_t}(\tau)$ multiplied by our decoder denoted $\mathcal{G}(z_t, \tau)$ which maps the tuple (z_t, τ) to a scalar, i.e.

$$P(z_t, \tau) = \exp(A_{z_t}(\tau) - B_{z_t}(\tau)\mathcal{G}(z_t, \tau)) \quad (7)$$

Contrary to the conventional affine model, the scalar functions $A_{z_t}(\tau)$ and $B_{z_t}(\tau)$ are indexed by the latent factor z_t , i.e., for every z_t we need to solve a coupled ODE to obtain the explicit forms of $A_{z_t}(\tau)$ and $B_{z_t}(\tau)$. This means there are as many functionals $A_{z_t}(\tau)$ and $B_{z_t}(\tau)$ as z_t . In other words, whenever a new latent variable z_t is encoded from the raw par swap rates, the corresponding functionals $A_{z_t}(\tau)$ and $B_{z_t}(\tau)$ are generated on the fly through an ODE solver such that the no-arbitrage condition is met for this instance of z_t .

We now look at the short rate and how to model it consistently. The instantaneous forward rate $f(t, \tau)$ in our framework (7) is given by

$$f(t, \tau) = -\frac{\partial \log P(t, \tau)}{\partial \tau} \quad (8)$$

$$= -\frac{\partial A_{z_t}(\tau)}{\partial \tau} + \frac{\partial B_{z_t}(\tau)}{\partial \tau}\mathcal{G}(z_t, \tau) + \frac{\partial \mathcal{G}(z_t, \tau)}{\partial \tau}B(\tau) \quad (9)$$

As the short rate $r(t)$ is defined as

$$r(t) = \lim_{\tau \rightarrow 0} f(t, \tau) \quad (10)$$

we conclude that $r(t)$ must be a function of the latent factors z_t . Next, we know that absence of arbitrage implies that bond prices can be written as a risk-neutral expectation of the bank account, i.e. (with a slight abuse of notation)

$$P(t, t + \tau) = \mathbb{E}_t^{\mathbb{Q}} \left[e^{-\int_t^{t+\tau} r(u) du} \right] \quad (11)$$

Here $P(t, t + \tau)$ denotes the zero-coupon bond price with absolute maturity $T = t + \tau$. Comparing (11) to (7) we furthermore conclude that z_t is a Markov process under the risk-neutral measure $\mathbb{Q}$. It is therefore natural to impose a dynamics on z_t and assume that $r(t)$ is a continuous transformation $\tilde{r}$ of z_t

$$dz_t = \mu(z_t)dt + \sigma(z_t)dW_t^{\mathbb{Q}} \quad (12)$$

$$r(t) = \tilde{r}(z_t) \quad (13)$$

where $\mu(z_t) \in \mathbb{R}^{2 \times 1}$ and $\sigma(z_t) \in \mathbb{R}^{2 \times 2}$ denote the drift and volatility of z_t , respectively, and $W_t^{\mathbb{Q}} \in \mathbb{R}^{2 \times 1}$ represents standard Brownian motions under the risk-neutral measure $\mathbb{Q}$. Equation (12) describes diffusion dynamics.

At first sight (13) might seem redundant as the short rate $r(t)$ can already be obtained through (10) and, more importantly, how to guarantee (10) and (13) yield the same value? We show at the end of this section that (13) is indeed necessary and serves as a “hard” constraint for the arbitrage-free decoding neural network $\mathcal{G}$ such that

$$r(t) = \lim_{\tau \rightarrow 0} f(t, \tau) = \tilde{r}(z_t) \quad (14)$$

is satisfied by construction.

With this knowledge at hand, we can derive a differential equation for P by using the no-arbitrage condition that the process $P(t, t + \tau)$ divided by the bank account

$$B(t) = \exp \left(\int_0^t \tilde{r}(z_u) du \right) \quad (15)$$

must be a martingale under $\mathbb{Q}$. This condition yields the following partial differential

equation (PDE)

$$-\tilde{r}(z_t)P(z_t, \tau) - \partial_\tau P(z_t, \tau) + (\nabla_z P(z_t, \tau))^T \mu(z_t) + \frac{1}{2} \text{Tr}[\sigma^T(z_t) H_z P(z_t, \tau) \sigma(z_t)] = 0 \quad (16)$$

Omitting the arguments of P for brevity, it is easy to verify that (7) satisfies the following relations

$$\partial_\tau P = (\partial_\tau A_{z_t}(\tau) - \mathcal{G}(z_t, \tau) \partial_\tau B_{z_t}(\tau) - B_{z_t}(\tau) \partial_\tau \mathcal{G}(z_t, \tau)) P \quad (17)$$

$$\nabla_z P = -B_{z_t}(\tau) \nabla_z \mathcal{G}(z_t, \tau) P \quad (18)$$

$$H_z P = (B_{z_t}^2(\tau) \nabla_z \mathcal{G}(z_t, \tau) \nabla_z^T \mathcal{G}(z_t, \tau) - B_{z_t}(\tau) H_z \mathcal{G}(z_t, \tau)) P \quad (19)$$

By substituting (17)-(19) into (16) we obtain

$$\begin{aligned} & -\tilde{r}(z_t)P - (\partial_\tau A_{z_t}(\tau) - \mathcal{G}(z_t, \tau) \partial_\tau B_{z_t}(\tau) - B_{z_t}(\tau) \partial_\tau \mathcal{G}(z_t, \tau)) P \\ & + (-B_{z_t}(\tau) \nabla_z^T \mathcal{G}(z_t, \tau) \mu(z_t)) P \\ & + \frac{1}{2} \text{Tr}[\sigma^T(z_t) (B_{z_t}^2(\tau) \nabla_z \mathcal{G}(z_t, \tau) \nabla_z^T \mathcal{G}(z_t, \tau) - B_{z_t}(\tau) H_z \mathcal{G}(z_t, \tau)) \sigma(z_t)] P = 0 \end{aligned} \quad (20)$$

which we rewrite as

$$\begin{aligned} & -\tilde{r}(z_t) - \partial_\tau A_{z_t}(\tau) + \mathcal{G}(z_t, \tau) \partial_\tau B_{z_t}(\tau) + B_{z_t}(\tau) \partial_\tau \mathcal{G}(z_t, \tau) - B_{z_t}(\tau) \nabla_z^T \mathcal{G}(z_t, \tau) \mu(z_t) \\ & + B_{z_t}^2(\tau) \frac{1}{2} \text{Tr}[\sigma^T(z_t) \nabla_z \mathcal{G}(z_t, \tau) \nabla_z^T \mathcal{G}(z_t, \tau) \sigma(z_t)] \\ & - B_{z_t}(\tau) \frac{1}{2} \text{Tr}[\sigma^T(z_t) H_z \mathcal{G}(z_t, \tau) \sigma(z_t)] = 0 \end{aligned} \quad (21)$$

We then group terms according to their order of $B_{z_t}(\tau)$

$$\begin{aligned} & -\tilde{r}(z_t) - \partial_\tau A_{z_t}(\tau) + \mathcal{G}(z_t, \tau) \partial_\tau B_{z_t}(\tau) \\ & + B_{z_t}(\tau) \left(\partial_\tau \mathcal{G}(z_t, \tau) - \nabla_z^T \mathcal{G}(z_t, \tau) \mu(z_t) - \frac{1}{2} \text{Tr}[\sigma^T(z_t) H_z \mathcal{G}(z_t, \tau) \sigma(z_t)] \right) \\ & + B_{z_t}^2(\tau) \frac{1}{2} \text{Tr}[\sigma^T(z_t) \nabla_z \mathcal{G}(z_t, \tau) \nabla_z^T \mathcal{G}(z_t, \tau) \sigma(z_t)] = 0 \end{aligned} \quad (22)$$

and separate the expression in two coupled ODEs

$$\partial_\tau B_{z_t}(\tau) = B_{z_t}(\tau) \underbrace{\frac{-\partial_\tau \mathcal{G}(z_t, \tau) + \nabla_z^T \mathcal{G}(z_t, \tau) \mu(z_t) + \frac{1}{2} \text{Tr}[\sigma^T(z_t) H_z \mathcal{G}(z_t, \tau) \sigma(z_t)]}{\mathcal{G}(z_t, \tau)}}_{\alpha_{z_t}(\tau)} + \underbrace{\frac{\tilde{r}(z_t)}{\mathcal{G}(z_t, \tau)}}_{\beta_{z_t}(\tau)} \quad (23)$$

$$\partial_\tau A_{z_t}(\tau) = B_{z_t}^2(\tau) \underbrace{\frac{1}{2} \text{Tr}[\sigma^T(z_t) \nabla_z \mathcal{G}(z_t, \tau) \nabla_z^T \mathcal{G}(z_t, \tau) \sigma(z_t)]}_{\gamma_{z_t}(\tau)} \quad (24)$$

with initial conditions $A_{z_t}(\tau = 0) = B_{z_t}(\tau = 0) = 0$. This system of coupled ODEs can be absorbed to a single ODE by introducing the vector $X_{z_t}(\tau) = (A_{z_t}(\tau), B_{z_t}(\tau))^T$ such that the no-arbitrage PDE in (16) can finally be expressed as the following ODE for each value of z_t coming from the encoding network

$$\partial_\tau X_{z_t}(\tau) = \begin{pmatrix} 0 & \gamma_{z_t}(\tau) \\ 0 & 0 \end{pmatrix} (X_{z_t}(\tau) \odot X_{z_t}(\tau)) + \begin{pmatrix} 0 & 0 \\ 0 & \alpha_{z_t}(\tau) \end{pmatrix} X_{z_t}(\tau) + \begin{pmatrix} 0 \\ \beta_{z_t}(\tau) \end{pmatrix} \quad (25)$$

where $X_{z_t}(\tau = 0) = (0, 0)^T$ and $\odot$ represents element-wise multiplication. By solving the ODE in (25) using fourth-order Runge-Kutta with 3/8 rule and applying (7) to the solved $A(\tau)$ and $B(\tau)$, we obtain the zero-coupon bond prices $P(z_t, \tau)$ used to compute the estimated swap rates $\tilde{S}$ in (3).

All that remains now is to explicitly verify that the limit (14) holds. From (9) we have

$$\lim_{\tau \rightarrow 0} f(t, \tau) = \lim_{\tau \rightarrow 0} \left(-\frac{\partial A_{z_t}(\tau)}{\partial \tau} + \frac{\partial B_{z_t}(\tau)}{\partial \tau} \mathcal{G}(z_t, \tau) + \frac{\partial \mathcal{G}(z_t, \tau)}{\partial \tau} B_{z_t}(\tau) \right) \quad (26)$$

We furthermore note that (23) and (24) given $\tau = 0$ yield boundary conditions for the first-order derivatives of $B_{z_t}(\tau)$ and $A_{z_t}(\tau)$

$$\partial_\tau B_{z_t}(\tau = 0) = \frac{\tilde{r}(z_t)}{\mathcal{G}(z_t, \tau = 0)} \quad (27)$$

$$\partial_\tau A_{z_t}(\tau = 0) = 0 \quad (28)$$

and since $B_{z_t}(\tau = 0) = 0$ we arrive at

$$\lim_{\tau \rightarrow 0} f(t, \tau) = \tilde{r}(z_t) \quad (29)$$

as advertised.

2.3 Parameter modeling

The last step remaining before a practical implementation is to parametrize (12) and (13). We start by writing (12) explicitly for a two factor model

$$\begin{pmatrix} dz_1(t) \\ dz_2(t) \end{pmatrix} = \underbrace{\begin{pmatrix} \mu_1(z_t) \\ \mu_2(z_t) \end{pmatrix}}_{\mu} dt + \underbrace{\begin{pmatrix} \sigma_1(z_t) & 0 \\ \rho(z_t)\sigma_2(z_t) & \sqrt{1-\rho^2(z_t)}\sigma_2(z_t) \end{pmatrix}}_{\sigma} \begin{pmatrix} dW_1^{\mathbb{Q}}(t) \\ dW_2^{\mathbb{Q}}(t) \end{pmatrix} \quad (30)$$

We remind the reader that $z_t \equiv (z_1(t), z_2(t))$. The parameters governing this stochastic differential equation can be modelled with neural networks. For the drift μ we introduce a neural network $\mathcal{K}$ that models it as a mean-reverting process by performing the linear mapping

$$\mathcal{K} : \begin{pmatrix} z_1(t) \\ z_2(t) \end{pmatrix} \rightarrow \begin{pmatrix} \mu_1(z_t) \\ \mu_2(z_t) \end{pmatrix} = \mathbf{M} \begin{pmatrix} z_1(t) \\ z_2(t) \end{pmatrix} + \mathbf{N} \quad (31)$$

where $\mathbf{M}$ and $\mathbf{N}$ are 2×2 and 2×1 real matrices, respectively. Looking at the volatility matrix σ , we construct a neural network for determining σ_1 , σ_2 and ρ . Dubbed $\mathcal{H}$, it has dimensions $(2, 4, 3)$ and performs the mapping

$$\mathcal{H} : \begin{pmatrix} z_1(t) \\ z_2(t) \end{pmatrix} \rightarrow \begin{pmatrix} \log(\sigma_1) \\ \log(\sigma_2) \\ \text{atanh}(\rho) \end{pmatrix} \quad (32)$$

where the transformations ensure the positivity of the volatilities and the boundedness of the correlation. We transform them back to volatilities and correlation afterwards.

The risk-free rate $\tilde{r}$ in (13) is also modelled by constructing a neural network $\mathcal{R}$ with dimensions $(2, 4, 1)$ that performs the mapping

$$\mathcal{R} : \begin{pmatrix} z_1(t) \\ z_2(t) \end{pmatrix} \rightarrow \tilde{r} \quad (33)$$

The activation function in the latent layer is the centered soft step function defined in (5) for both $\mathcal{H}$ and $\mathcal{R}$ and all bias terms are set to zero.

2.4 Model Estimation

The encoder-decoder in Figure 1 is estimated by adjusting the weights A_1 of the encoder and the networks $\mathcal{G}$, $\mathcal{K}$, $\mathcal{H}$ and $\mathcal{R}$ of the decoder such that the ℓ_2 loss f between the input $S_t(\tau)$ and output $\tilde{S}_t(\tau)$ is minimized across all observation dates t . Denoting the total set of training parameters by Ω , the steady state solution therefore satisfies

$$\hat{\Omega} = \min_{\Omega} f(\Omega) \quad (34)$$

$$= \min_{\Omega} \frac{1}{N} \sum_{t=1}^N \sum_{\tau} \left(S_t(\tau) - \tilde{S}_t(\tau; \Omega) \right)^2 \quad (35)$$

where N is the total number of observations. The numerical solution to this minimization problem is found by stochastic gradient descent using the ADAM optimizer (Kingma and Ba, 2015) with a learning rate of 10^{-3} and training batch size of 32 for 1 000 epochs. The backpropagation through the ODE solver follows the methodology by Chen et al. (2018).

2.5 Results

We employ a data set similar to that used by Andreasen (2023) consisting of liquid par swap rates for the maturities in (1) for the 9 currencies

$$\text{DKK, EUR, GBP, SEK, NOK, USD, AUD, CAD, JPY} \quad (36)$$

The rates are observed monthly over a 13-year period from January 2010 to December 2022, yielding a total of 1412 curves. Examples of the curves are illustrated in Figure 2.

We follow Sokol (2022) and Andreasen (2023) and assume that the currencies can be represented by the same model, meaning that we weigh all swap curves equally during training of the encoder-decoder. Combining curves from multiple currencies into a single data set is a clever way to obtain sufficient training data and, as remarked by Sokol (2022), enhances the encoder-decoder’s ability to accurately represent currency-specific curves.

2.6 Performance

The resulting in-sample root mean square errors (RMSE) in basis points (bps) are presented in Table 1.

The RMSEs range between 9.59 bps and 18.37 bps across currencies with an average just below 13 bps. For a fairly simple model (linear encoder, two factors, constant

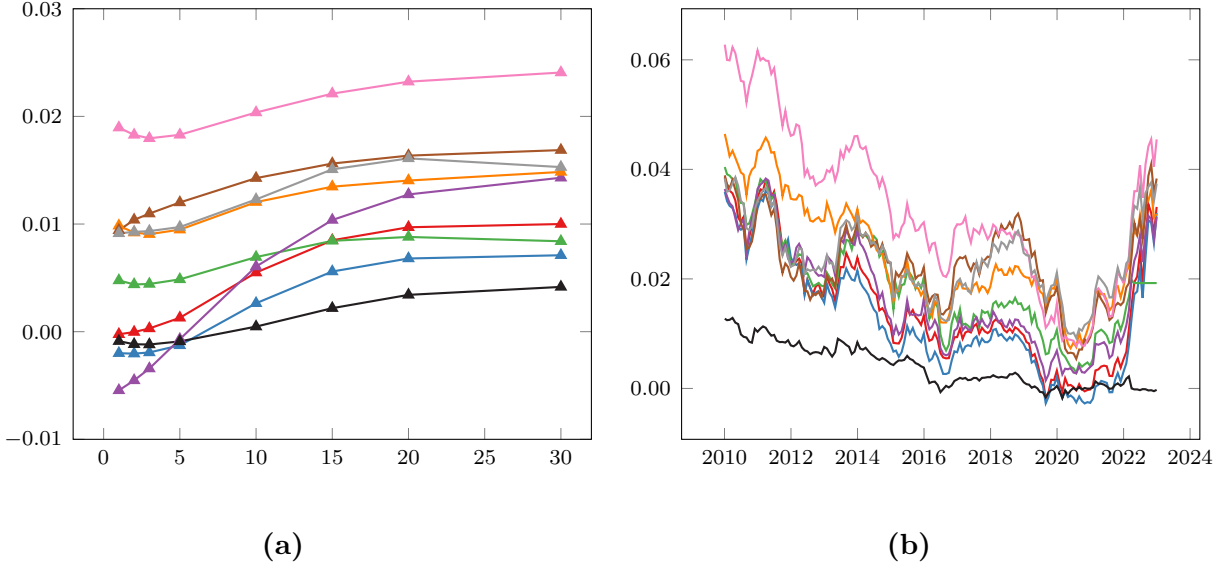


Figure 2: Swap rates for the currencies DKK ($\blacktriangle$), EUR ($\blacktriangle$), GBP ($\blacktriangle$), SEK ($\blacktriangle$), NOK ($\blacktriangle$), USD ($\blacktriangle$), AUD ($\blacktriangle$), CAD ($\blacktriangle$) and JPY ($\blacktriangle$) for (a) the maturities $\{1Y, 2Y, 3Y, 5Y, 10Y, 15Y, 20Y, 30Y\}$ on the 30th August 2016 and (b) a 10 year maturity over the period 4th January 2010 – 30th December 2022.

parameters across both time and currencies), this is very decent accuracy. [Andreasen \(2023\)](#) considers the same time period and yield curves (with the exception that CZK and SGD are included, while SEK is excluded) and reports an average RMSE of 10 bps for a two-factor model. As explained, the main difference between the two approaches lies in the treatment of the no-arbitrage condition: Our model is completely arbitrage-free by construction (albeit with added computational cost stemming from having to solve ODEs), while Andreasen’s isn’t (enabling him to achieve a better fit).² Rather, he applies a no-arbitrage check by calculating the zero-coupon bond Sharpe ratios implied by the trained network. For the model to be arbitrage-free, they must all be zero – which they aren’t quite; values range between -0.15 and 0.1. A representative calculation shows that our model yields values smaller than 10^{-5} in absolute terms across all currencies as illustrated in Figure 3.

Notice how the results from the two approaches corroborate each other; there is information not only in the similarities but also in the differences. Looking at Andreasen’s results on a stand-alone basis, it would not (at least to us) be clear whether the magnitude of the Sharpe ratios should concern us. The results from our model tells us that they shouldn’t. On the other hand, looking at our results on a stand-alone basis, one might

²In subsequent unpublished work, Andreasen includes the Sharpe ratio term as a penalty or regularization term in the loss function when training the model.

Currency	RMSE
DKK	12.76 bps
EUR	18.37 bps
GBP	10.71 bps
SEK	15.45 bps
NOK	9.59 bps
USD	11.75 bps
AUD	11.94 bps
CAD	10.44 bps
JPY	12.93 bps
Average	12.93 bps

Table 1: Steady-state in-sample RMSE of the encoder-decoder architecture shown in Figure 1.

worry that the hard enforcement of the no-arbitrage condition could significantly diminish the fit to data, potentially causing us to miss arbitrages or good deals. The results from Andreassen’s model tell that it doesn’t.

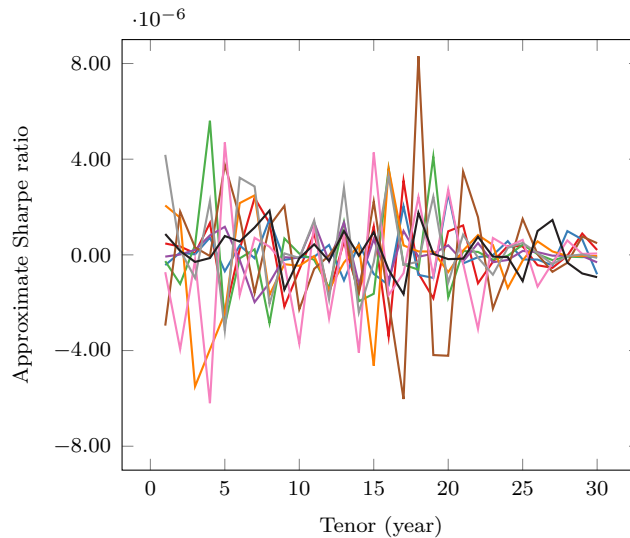


Figure 3: Approximate Sharpe ratio for the currencies DKK (—), EUR (—), GBP (—), SEK (—), NOK (—), USD (—), AUD (—), CAD (—) and JPY (—) on the 28th December 2014.

2.6.1 Relation to Principal Component Analysis

As our encoder is merely a linear transformation, we can directly compare our latent factors to the principal components simply by projecting the weight matrix of the encoder on to the space spanned by the principal components. We choose the eight components

with the largest eigenvalue illustrated in Figure 4.

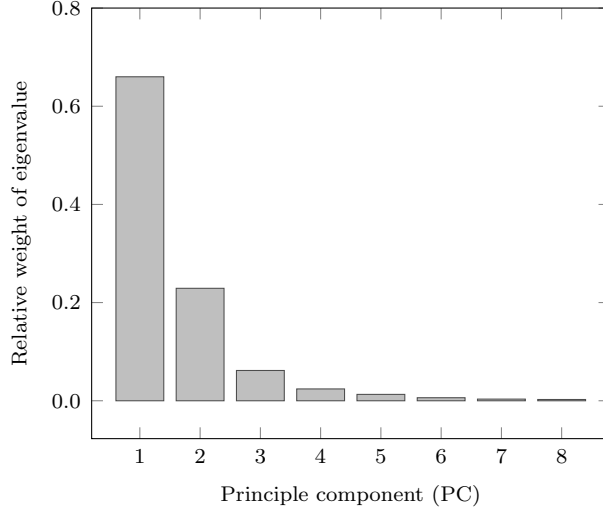


Figure 4: The relative weights of eigenvalues are sorted in a decreasing order. The weights are negligible for the fourth principal component and beyond.

To facilitate the comparison, we start by writing a given encoding weight matrix A_1 as

$$A_1 = \begin{pmatrix} W_1 \\ W_2 \end{pmatrix} \quad (37)$$

where $W_1 \in \mathbb{R}^{1 \times 8}$ and $W_2 \in \mathbb{R}^{1 \times 8}$ are the first and the second rows of A_1 , respectively. The eigenvectors $\{V_i\}_{i=1}^8$ obtained from PCA are grouped into a matrix following a decreasing order of the corresponding eigenvalues,

$$V = \begin{pmatrix} V_1 & V_2 & \dots & V_8 \end{pmatrix} \quad (38)$$

where $V_i \in \mathbb{R}^{8 \times 1}$ for $i = 1, 2, \dots, 8$. The projection coefficient c_{ij} of weight W_i on eigenvector V_j is then merely given by the dot product $c_{ij} = W_i \cdot V_j$ and its square is the corresponding projection weight $w_{i,j} \equiv c_{i,j}^2 \in [0, 1]$. In Table 2 we present the projection weights, where it is immediately clear that the majority of the weights are concentrated on the two principal components with the largest singular values. In Figure 5 we show the two latent factors z_1 and z_2 from the encoder.

	PC ₁	PC ₂	PC ₃	PC ₄	PC ₅	PC ₆	PC ₇	PC ₈
Latent Factor 1	12.32 %	77.13 %	2.65 %	0.56 %	5.39 %	1.73 %	0.20 %	0.01 %
Latent Factor 2	41.75 %	44.47 %	2.20 %	1.33 %	3.92 %	5.59 %	0.64 %	0.09 %

Table 2: The projection weights of the encoded two factors on all eight eigenvectors.

2.6.2 Risk-Neutral Parameters

In Figure 6 we present the extracted short rate r , volatilities σ_1 and σ_2 , correlation ρ and drifts μ_1 and μ_2 from the encoder-decoder for all currencies. Interestingly, the behavior of the short rate r is in line with our intuitive expectations: The EUR and DKK short rates are almost identical as the latter is pegged to the former. Furthermore, from 2010 to 2020 most of the economies were in a low-rate environment, meaning that rates either went down or remained unchanged. The COVID-breakout in 2020 was detrimental to global economies as businesses were forced to shut down and consumer spendings were cut off almost instantly. To boost the economy and encourage spendings, central banks had to lower rates, yielding an extremely low-rate environment globally. Ever since 2022, the inflation pressure has forced most central banks to hike rates with JPY being the exception due to a fight against deflation. The volatility of JPY is also much more stable over the time period relative to other currencies, indicating the uniqueness of the monetary policy of Japan. Another interesting thing to notice is that the volatilities of the two latent factors had a significant drop when central banks started hiking rates, and their correlation is approaching one across multiple currencies.

3 Conclusion

Yield curve modeling has traditionally been dominated by parametric representations from the Nelson-Siegel family (Nelson and Siegel, 1987) or classical linear principal component analysis. The extension to nonlinear representations using autoencoders was pioneered by Kondratyev (2018), signifying the close relationship between encoder-decoders and principal component analysis. In this work we explicitly corroborate this relationship, which stems from their mutual dimension reduction ability.

By decoupling the encoding and decoding stages, we furthermore demonstrate that it is possible to impose the no-arbitrage condition as a hard constraint during the latter. Drawing inspiration from exponential affine yield curve models, we design a simple encoder-decoder architecture that is inherently arbitrage-free and able to produce a con-

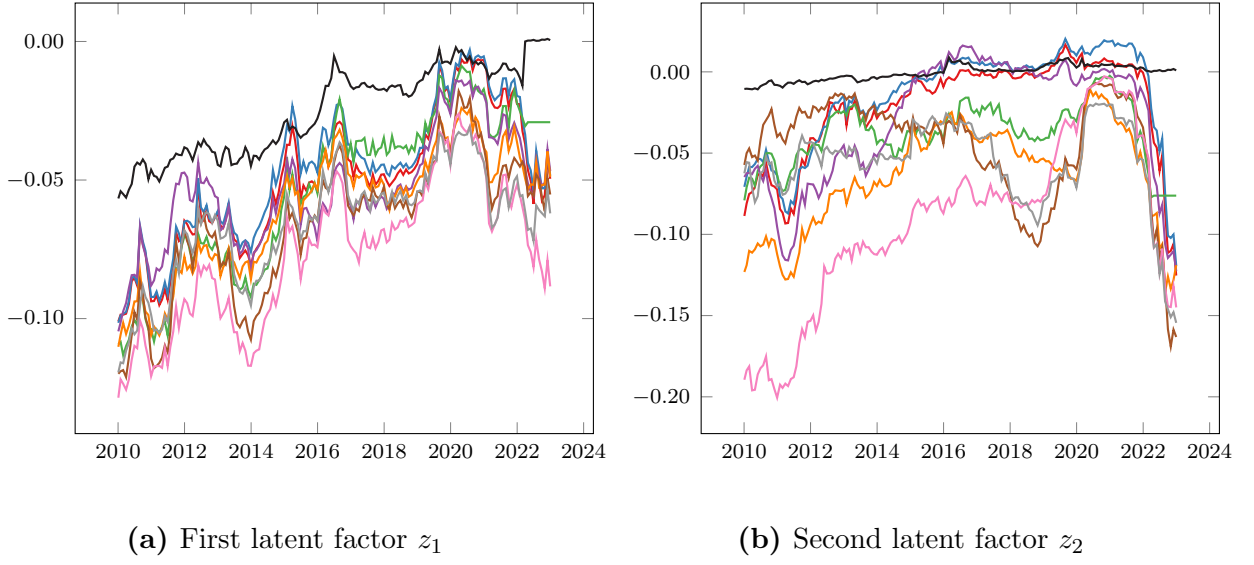
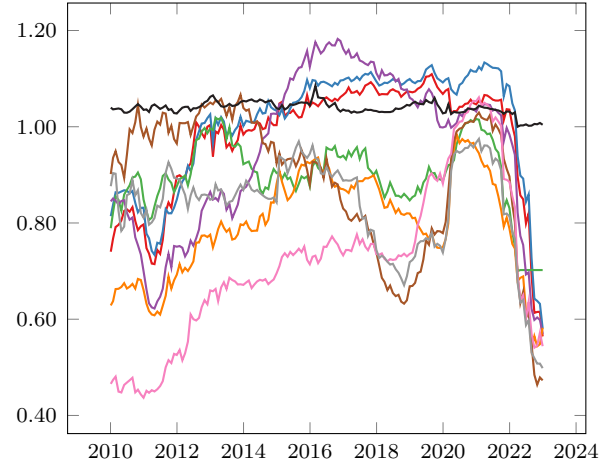


Figure 5: Encoded latent factors for the currencies DKK (—), EUR (—), GBP (—), SEK (—), NOK (—), USD (—), AUD (—), CAD (—) and JPY (—).

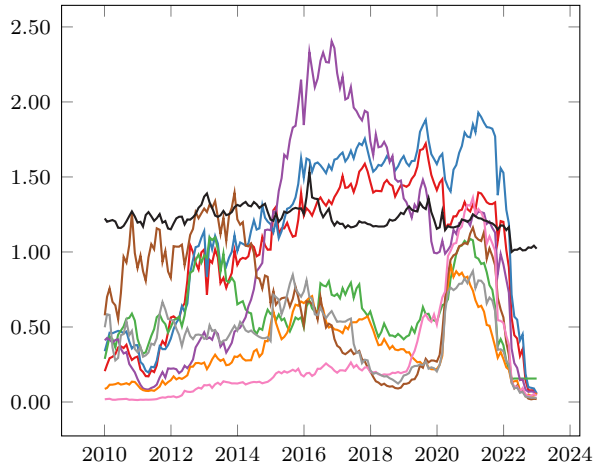
tinuous representation while achieving accuracy comparable to [Andreasen \(2023\)](#). While a detailed quantitative comparison with the recent work of [Lyashenko et al. \(2025\)](#) is not yet feasible due to limited data availability, our results underscore the potential of integrating classical financial theory with modern machine learning techniques.



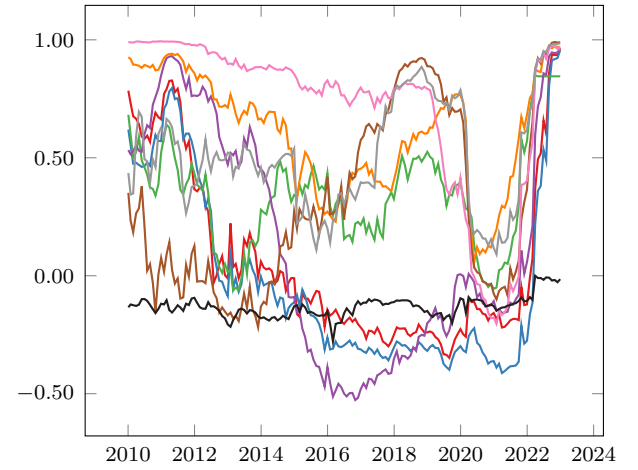
(a) r



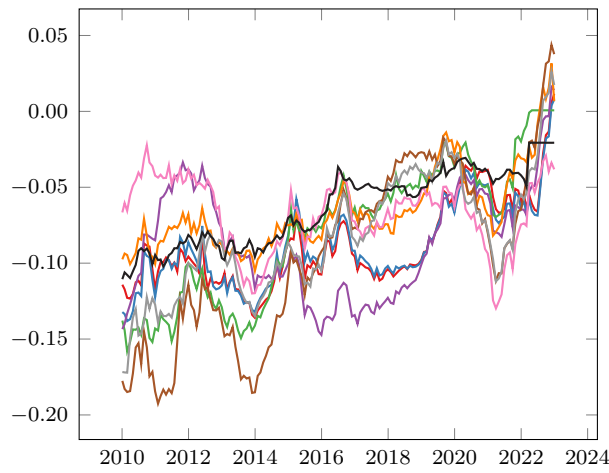
(b) σ_1



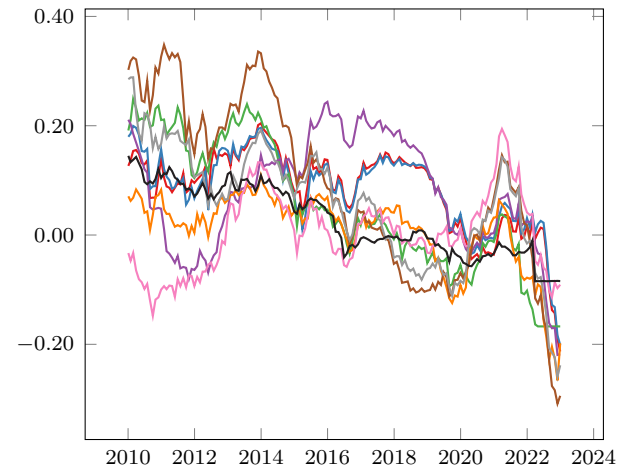
(c) σ_2



(d) ρ



(e) μ_1



(f) μ_2

Figure 6: Risk-neutral parameters of par swap rates for the currencies DKK (—), EUR (—), GBP (—), SEK (—), NOK (—), USD (—), AUD (—), CAD (—) and JPY (—).
16

References

- ANDREASEN, J. (2023): “Decoding the Autoencoder,” *Wilmott*, 2023.
- BJÖRK, T. (2009): *Arbitrage Theory in Continuous Time*, Oxford University Press, third ed.
- CHEN, R. T. Q., Y. RUBANOVA, J. BETTENCOURT, AND D. K. DUVENAUD (2018): “Neural Ordinary Differential Equations,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., vol. 31.
- DUFFIE, D. AND R. KAN (1996): “A Yield-Factor Model of Interest Rates,” *Mathematical Finance*, 6, 379–406.
- HENRY-LABORDERE, P. (2008): *Analysis, Geometry, and Modeling in Finance: Advanced Methods in Option Pricing*, Chapman & Hall/CRC, first ed.
- JIANG, C., Z. HUANG, T. PEDAPATI, P.-Y. CHEN, Y. SUN, AND J. GAO (2024): “Network Properties Determine Neural Network Performance,” *Nature Communications*, 15, 48069.
- KINGMA, D. AND J. BA (2015): “Adam: A Method for Stochastic Optimization,” in *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA.
- KONDRATYEV, A. (2018): “Curve Dynamics with Artificial Neural Networks,” *Risk*.
- LYASHENKO, A., F. MERCURIO, AND A. SOKOL (2025): “Auto-Encoding Term-Structure Models,” *Risk*.
- MINAEI, S., Y. BOYKOV, F. PORIKLI, A. PLAZA, N. KEHTARNAVAZ, AND D. TERZOPOULOS (2022): “Image Segmentation Using Deep Learning: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44, 3523–3542.
- NELSON, C. R. AND A. F. SIEGEL (1987): “Parsimonious Modeling of Yield Curves,” *The Journal of Business*, 60, 473–489.
- SOKOL, A. (2022): “Autoencoder Market Models for Interest Rates,” Working paper on SSRN.